MASTER'S MODULE · AGENTIC AI · WINTER SEMESTER

Sandboxing & Guardrails

Making a capable agent safe to run — isolation below, behavioural limits above.

Unit 07 · Part 1 of 2 · Session 10 · Week 10

Prof. Dr. Hauke Schramm · Department of Computer Science · Kiel University

Today's agenda

1. **Threat framing** what agents add to classical software risk
2. **Sandboxing** isolation levels, filesystem, network, credentials
3. **Guardrails** validation, action policies, approvals, budgets
4. **Defense in depth** layering, failure stories, lab tie-in

Where we are

Unit 07, part 1 — from capability to operational safety

Units 01-06 — we built capability: the loop, tools, prompting, orchestration, memory, RAG

Last week (S09) — retrieval and RAG: agents now act on text they did not write and nobody reviewed

The new problem — a capable agent with real tools can now do real damage

This unit — operating agents responsibly: containment today, observability next week

GUIDING QUESTION

We cannot predict every action the agent will take. Which properties can we still enforce?

1

Threat framing

What agents add to classical software risk

The core risk equation

An agent's tools define its blast radius.

The worst case of a run is the worst combination of effects its tools permit — independent of what the model intends. Risk assessment therefore starts from the tool inventory and its privileges, not from the model.

What agents add to ordinary software risk

Additional challenges on top of classical security — not a replacement for it

Learned decision policy — the policy lives in weights; you cannot review it, only test it statistically

Sampled behaviour — the same input can yield different actions; failures often vanish on retry

Instruction ambiguity — natural-language data and instructions share one channel (Greshake et al., 2023)

Compounding steps — a small early deviation steers every later action in the run

Consequence — verify deterministic parts and invariants; evaluate the model statistically; constrain effects

THE SHIFT

Classical verification stays. Added: statistical evaluation of model behaviour, and hard limits on the cost of behaviour gone wrong.

Classical practice vs what agents add

Your software-engineering instincts remain necessary — they are no longer sufficient

Classical software (still applies)

- logic fixed in code; reviewable, testable
- injection is a known input-validation problem
- most failures reproduce; concurrency does not
- tests sample a bounded input space
- verify invariants, then deploy

LLM agents (in addition)

- decision policy learned, sampled anew per run
- any read text can instruct, in plain language
- failures often vanish on retry
- action space effectively unbounded
- also: constrain effects, evaluate statistically

2

Sandboxing

Damage limitation: bounding what a wrong action can reach

What a sandbox is — and is not

Definition — an execution environment that bounds the side effects available to the agent

Working assumption — the agent will eventually do the wrong thing; design for that day

Bounds, not prevents — isolation changes neither error probability nor authority inside the box; it caps possible impact

Default deny — grant capabilities explicitly; everything not granted is unreachable

Five dimensions — filesystem, network, credentials, compute, wall-clock time

OLD WISDOM

Least privilege and fail-safe defaults date to Saltzer & Schroeder (1975) — agents just made them urgent again.

Isolation levels: boundary, exposure, overhead

Four standard boundaries — what each exposes depends on its configuration

Level	Boundary	Exposed trusted components	Overhead (config.-dependent)	Example use
Process + seccomp	OS process, filtered syscalls	shared host kernel, allowed syscalls	lowest: no extra kernel	trusted helpers, light limits
Container	namespaces, cgroups	shared host kernel, runtime, mounts	low: shared-kernel start	many agent tool runtimes
MicroVM	guest kernel behind a hypervisor	hypervisor, device model, host	extra kernel and memory per VM	hostile code, e.g. serverless
WebAssembly	linear memory plus host imports	runtime and every host import	in-process; runtime-dependent	plugins, bounded compute

Numbers only with their setup: ~125 ms boot is Firecracker's own benchmark (Agache et al., 2020). Wasm memory safety is not deterministic or domain-safe execution.

Filesystem scoping and ephemeral runs

Workspace mounts — the agent sees one project directory — never your home directory

Read/write split — broad read access is sometimes fine; write access stays narrow

No secrets in tree — no .env files, SSH keys, or browser profiles inside the mount

Ephemeral by default — fresh environment per run limits local persistence — mounts, exported artifacts and external services persist

Three different mechanisms — teardown resets the box; rollback reverts data; already-sent external actions need compensation

Copy-on-write — let the agent mutate freely, then diff and apply changes selectively

THE RM -RF TEST

If the agent ran rm -rf / inside its environment right now — what, exactly, would be lost? Count the mounts.

Network egress and scoped credentials

Default-deny egress — outbound traffic only to an allowlist: APIs, package registries

Why egress matters — the network is the exfiltration channel for anything the agent was tricked into reading

Scoped tokens — read-only, single-repository, short-lived — never an admin key

No ambient secrets — environment variables leak into every subprocess the agent spawns

Proxy pattern — a gateway holds the credentials and must itself authorize each operation and destination

RULE OF THUMB

Hiding credentials reduces secret exposure. It does not stop misuse through the proxy — the gateway must authorize what the credential is used for.

Code-execution sandboxes

When the agent writes the code that runs

Why agents write code

Code is the universal tool: data analysis, file transforms, gluing APIs — one sandbox replaces fifty bespoke tools.

Untrusted by construction

Generated code may be wrong, or shaped by injected instructions. Treat it exactly like code submitted by a stranger.

The sandbox contract

No network by default, scoped filesystem, CPU and memory caps, hard wall-clock timeout, no host credentials.

Fresh every run

Ephemeral interpreter per execution limits local persistence. Shared volumes, caches and exported artifacts still carry state between runs — scope them too.

3

Guardrails

Behavioural limits: deciding which actions happen at all

Input and output guardrails

Checks at the boundaries of every model call

Input side

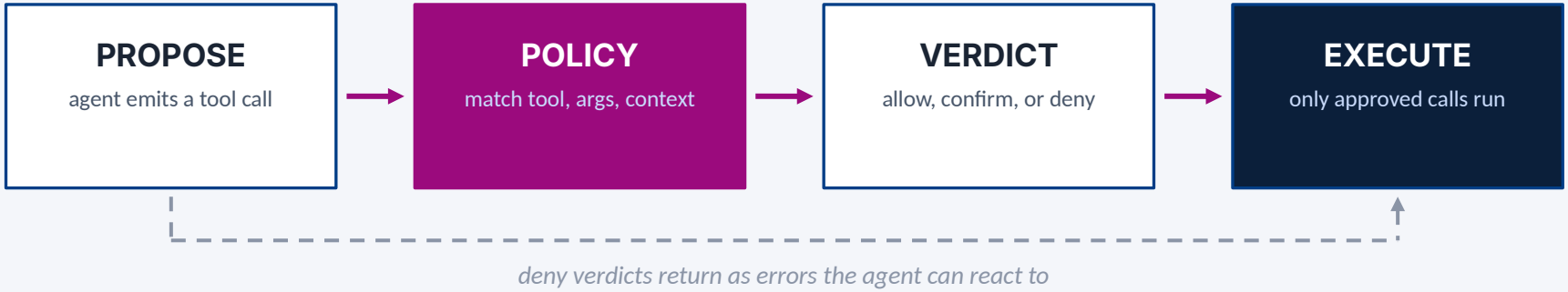
- validate the request: in scope, well-formed
- screen retrieved content before it enters context
- mark untrusted text as data, not instructions
- cap size and type of tool results

Output side

- schema enforcement: validate, then re-ask on failure
- tool-argument checks: paths, URLs, query shape
- groundedness: claims must cite a retrieved source
- content filters before anything reaches the user

Action policies: the decision per tool call

Every proposed action passes a policy layer the model cannot bypass



- Allow list: routine, reversible calls run without friction — but argument checks (path, URL) still apply.
- Confirm list: consequential calls pause for a human — writes, sends, purchases; exactly the shown args run.
- Deny list: out-of-policy calls are blocked with a structured error, not silently dropped.

Human-in-the-loop approval gates

What gets gated — irreversible or costly actions: send, delete, deploy, pay, publish

The reversibility test — can this action be cheaply undone? If not, a human signs off

Approval UX — show the exact action, arguments, and context — not 'agent wants to proceed'

Approval fatigue — too many prompts breed rubber-stamping; the gate silently decays

Earned autonomy — widen the allow list per tool and per environment as trust accumulates

MEASURING A GATE

Monitor rejection rate together with exposure, sampled review quality and missed violations. A zero rejection rate alone proves nothing — the proposed actions may simply be correct.

A policy-checked tool wrapper

Enforcement in code, outside the model; POLICY maps principal and tool name to allow / confirm / deny (Lab 10)

guard.py

```
def authorize(principal, tool, args, task_scope):
    verdict = POLICY[principal].get(tool, "deny")  # default
    if "path" in args:                  # file tools
        p = Path(args["path"]).resolve()  # .. and symlinks
        if not p.is_relative_to(task_scope.workspace):
            return "deny"
    if "url" in args:                   # fetch / send tools
        if host(args["url"]) not in task_scope.allowed_hosts:
            return "deny"
    return verdict                  # allow | confirm | deny
def guarded(principal, tool, args, task_scope):  # agent-proof
    v = authorize(principal, tool, args, task_scope)
    if v == "deny": return ToolError("blocked by policy")
    if v == "confirm" and not ask_human(tool, args):
        return ToolError("rejected by user")
    return TOOLS[tool].run(**args)  # approved args, unchanged
```

Budget caps and kill switches

Cap every axis — tokens, money, wall-clock time, loop steps, calls per tool

Why caps — the S02 failure modes — retry loops and derailment — now burn real money

Degrade gracefully — near the cap: stop acting, summarise progress, report partial results

Kill switch — an out-of-band stop that does not depend on the agent cooperating

Safe interruption — define transaction boundaries; log each call as done or unclear; reconcile with the external system

Retries vs partial effects — idempotency supports safe retries; atomicity or compensation handles partial effects

TIMING

Set every budget before the first run, not after the first invoice.

4

Defense in depth

No single layer suffices — use diverse layers and analyse what they share

Four layers, each fallible in its own way

Each layer should catch what the one above lets through — unless they share a cause: one flawed policy, runtime or config

Model layer

System-prompt rules and refusal training. Cheapest and weakest: anything here can be talked around.

Guardrail layer

Validation, action policies, approval gates, budgets — enforced in code the model cannot rewrite. Only as good as the policy.

Sandbox layer

Isolation, scoped filesystem, egress control, minimal credentials. Bounds damage within its configuration; runtimes have CVEs too.

Oversight layer

Monitoring, alerts, humans watching, the kill switch. Catches what slipped through — if someone is actually looking.

How this fails in practice

Reported incidents, each with its documented effect, source and missing control — the causes differ

Incident	Reported effect	Source	Missing control
Coding agent, Jul 2025	production database deleted during a declared code freeze	affected founder, vendor statement	freeze was prompt text; no separate write credential
Destructive shell commands	coding agents wiping real directories	recurring user reports, 2024-25	no workspace mount, no confirm gate by default
Runaway spend	retry loops against paid APIs, found on the invoice	practitioner reports	no budget cap, no spend alert
EchoLeak, CVE-2025-32711	zero-click exfiltration from M365 Copilot via crafted email	Aim Security, Jun 2025; patched before known abuse	injection classifier and link filters bypassed
Amazon Q extension, Jul 2025	destructive prompt shipped in a released VS Code extension	vendor bulletin; no customer impact reported	supply-chain review of contributions

A prompt instruction is a request. Code and sandboxes enforce explicit properties under stated assumptions, not blanket guarantees. No single cause fits all cases.

Lab 10: build and attack your own guardrails

Plain-Python guardrails over an offline synthetic corpus; Ollama only for one optional cell (Parts A-F, 90-120 min)

Blast radius (Part A) — list the lab's tool inventory; a disposable scratch workspace is the only directory tools may touch

Action policy (Part B) — wrapper returning allow / confirm / deny, enforced in code; unlisted tools fail closed

Argument guardrails (Part C) — workspace path allowlist with traversal tests (R4); output filter for secrets and PII

Gate, budget, kill switch (Part D) — approval gate, step and per-tool caps, out-of-band stop that returns a partial result

Red team (Part E) — a fetched synthetic page carries an injected instruction; record which layer catches each attack (R2)

DELIVERABLE (R1)

A one-page threat sheet: each tool's effect, its blast radius and the lowest layer that contains it. R3 argues confirm lists for dev vs production.

TAKEAWAYS

What you should remember

- **Blast radius —** tools and their privileges define the worst case, not model intent
- **Added risk —** learned, sampled policies and instruction ambiguity on top of classical software risk
- **Sandboxing —** default-deny isolation: filesystem, egress, credentials; ephemeral runs limit local persistence only
- **Guardrails —** validation, argument-level action policies, approval gates, budgets, kill switch
- **Enforcement in code —** prompts request; code and sandboxes enforce explicit properties, under assumptions
- **Defense in depth —** diverse layers, and an analysis of the common-mode failures they share

NEXT SESSION Observability — tracing and debugging nondeterministic agent runs.

Reading & references

- Saltzer, J. H., Schroeder, M. D. (1975). The Protection of Information in Computer Systems. Proceedings of the IEEE 63(9).
  Least privilege and fail-safe defaults — fifty-year-old principles, newly urgent.
- Agache, A. et al. (2020). Firecracker: Lightweight Virtualization for Serverless Applications. NSDI 2020.
  The microVM design point: VM-strength isolation at near-container overhead.
- Haas, A. et al. (2017). Bringing the Web up to Speed with WebAssembly. PLDI 2017.
  Memory-safe in-process sandbox; the capability-based system interface came with WASI (2019).
- Greshake, K. et al. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173.
  Why retrieved data must be treated as potential control input — deepened in S13.
- OWASP GenAI Security Project (2026). OWASP Top 10 for LLM Applications, 2026 edition; and OWASP Top 10 for Agentic Applications (2026).
  See LLM03 Excessive Agency (up from #6 in 2025) and, in the agentic list, Tool Misuse and Agent Identity & Privilege Abuse.
- Rebedea, T. et al. (2023). NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications. arXiv:2310.10501.
  A programmable-rails framework: policies as code around the model.
- Anthropic (2024). Building Effective Agents. Slide 22 incidents: Microsoft advisory CVE-2025-32711 (Jun 2025); AWS bulletin on Amazon Q Developer 1.84.0 (Jul 2025); Replit statement (Jul 2025).